

A Data Collection System to Find the Best Immediate Route for Motorcyclists

Dawson White

Parallel and Distributed Computing

Spring 2025/Section 01

March 13, 2025

Abstract

The both exciting and dangerous nature of motorcycling has led me to design a distributed messaging system (DMS) made up of a drone that gathers data pertaining to roads in the immediate area and relays it back to motorcyclists which allows them to make an informed decision on the best route to take. This report covers the architecture, entities, and data sent and received within this system.

Introduction

This paper described the creation and mindset behind a DMS that provides accurate and personalized navigational information for motorcyclists. Related domains are looked into as they help shine light on any issues that may be shared between my domain and others that are similar. The purposes or goals will highlight what this distributed system is meant to accomplish and how this will make the motorcycling hobby safer and more fun for those involved. A high-level overview is provided in this section of the report that explains how the system works to provide the solutions to the problems spoken of in previous sections. I then explore what makes my particular DMS unique, especially within my chosen domain. Of course, with any system there are some limitations and assumptions made, and I will go over how this molded my final design.

The second half of this report will begin with a couple figures/diagrams detailing a visual representation of the entities, types of data, and the flow of the data. This will include the drone is what data it collects under what circumstances, where the motorcyclists send the data they have received from the drone, as well as what each entity chooses to do with this information and how they interpret it.

Problem Domain

The problem domain for this DMS is directly related to motorcycles and specifically the navigation aspect of this topic. The problem comes when motorcyclists are forced to go in blindly to find a route that best fits their personal preferences.

Related Problems

Problems adjacent to my chosen domain include GPS issues for car drivers. An article by the Swiss National Science Foundation summarizes how GPS systems do not allow users to customize exactly what they want in a route which leads to traffic jams and often increased pollution since GPS systems prioritize almost only the fastest route (Swiss National Science Foundation, 2024). My DMS is similar as it

addresses the issue of the lack of personal preference when picking what a user wants as opposed to constantly focusing primarily on finding the fastest route. My system diverges as it focuses on a motorcyclist's needs. They may want a "curvier" or group friendly route that is rarely wanted by others.

Purpose and Goals

The goal of my DMS is to allow motorcyclists to make an accurate and personalized decision on their routes. This will not only be safer, but it will allow users to pick a route that best fits their preferences based on the images and recommendations coming back from the drone. This may include finding a path that looks to be more exciting, less miles, or backroads only.

Methodologies/Approaches

This DMS approaches the purposes listed in the above section by utilizing a drone that flies up and autonomously performs an overview in the user's chosen area. This data is then sent to the user's phone along with recommendations on which route to take based on the user's expressed preferences. The user can then parse this data and customize or simply pick one of the routes the drone has recommended. The user then has the option to sync this data with motorcyclists around in the case of a group ride and any motorcycles with the appropriate software (built-in GPS system, autonomous driving).

To summarize, these methodologies solve the problems presented as it allows users to stay stationary in a safe area while they carefully decide the optimal route as opposed to figuring it out while moving or running into something unexpected.

Qualitative Metrics

Metrics that support the uniqueness of my system are pure accuracy and timeliness. Current GPS systems look at road conditions that are very far away and could quickly change by the time the user reaches that area. Traditional systems struggle to provide live information that my DMS delivers signifying the accuracy in my system.

Constraints/Limitations

The limitations pertaining to my system primarily concern the drone's range and time to accurately assess the area. Since the drone is limited to range, a cheap drone would be extremely limited. A user would also be forced to wait an unknown amount of time for the drone to collect data.

Software Architecture

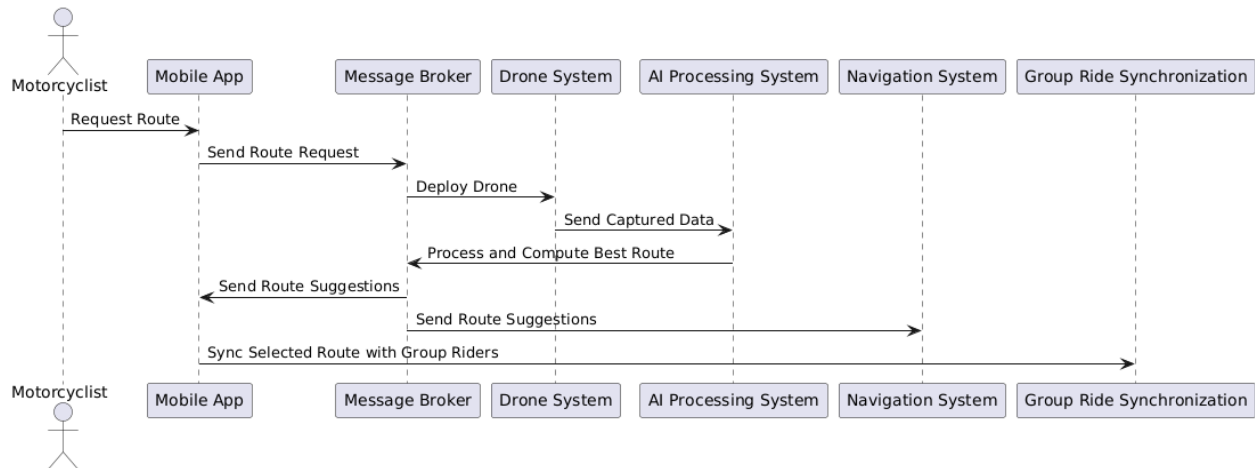


Figure 1: UML Representation of Data Collection System for Motorcyclists

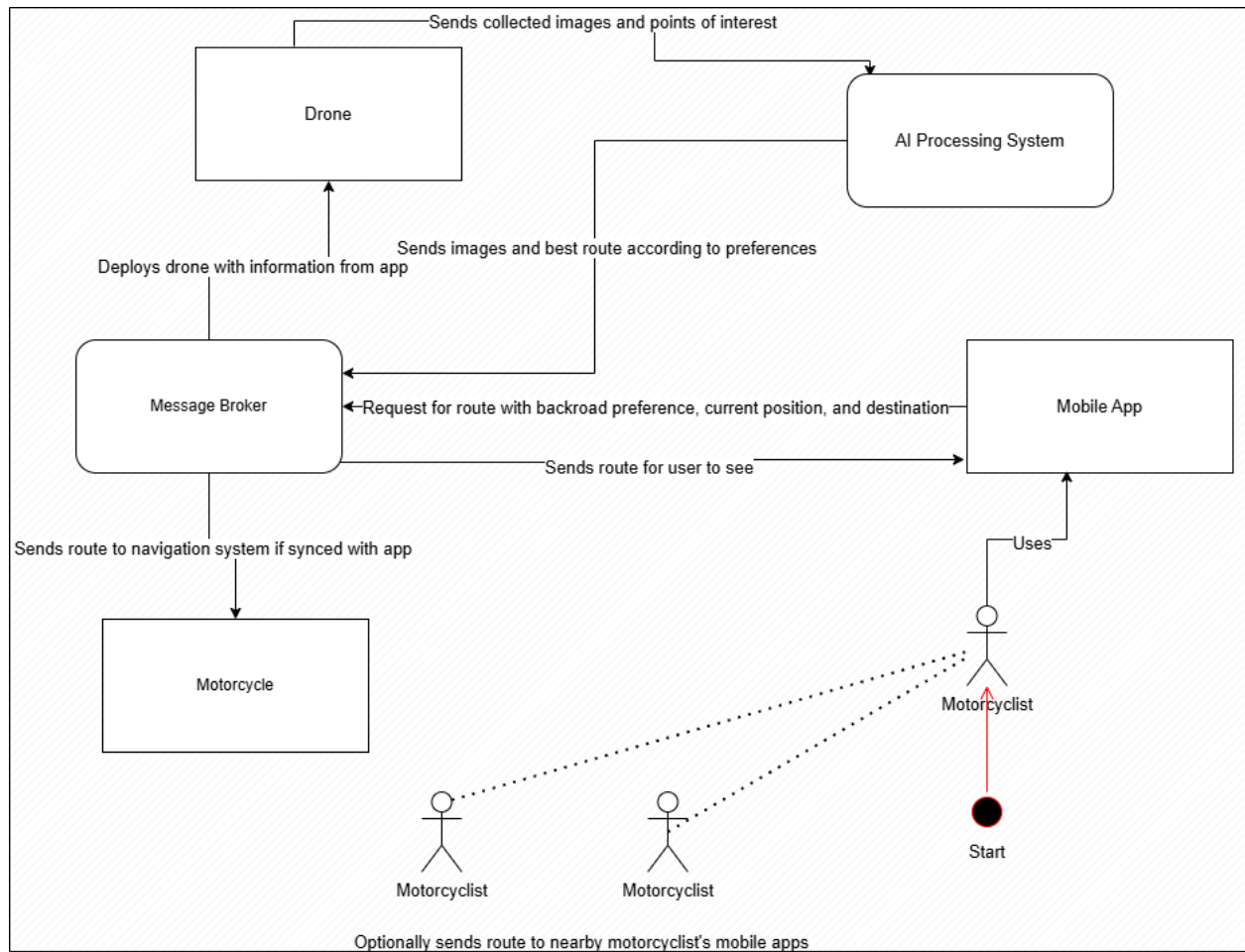


Figure 2: Model for Data Collection System Simulating a Request for a Strictly Backroads Route

Model Description

The two figures in the software architecture represent a visualization of how this DMS works as well as what data is shared between what entities. Figure 1 shows an UML model detailing an overview of how the distributed system works sequentially. Figure 2, on the other hand, uses a specific scenario of a motorcyclist wanting to find a route to a destination using only backroads implying the avoidance of any type of main road. I will be using figure 2 as my main reference to give a detailed description of the DMS.

The system is begun by a single motorcyclist that uses a mobile app on their phone that is used for sending a request to the drone as well as receiving the results. In this scenario, the motorcyclist chooses to set a preference for a route from his current location to an entered destination; however, only by backroads. The motorcyclist did not in this situation but could choose to enter additional preferences such as asking the drone to pay extra attention to ensure that these backroads are not backed up. This information gets sent to a message broker and then on to the drone itself. When the drone receives this information, it flies up and out as far as it can go towards the chosen destination while taking pictures and collecting data on the roads nearby. According to an article from Drone Pilot Ground School this range averages to be as far as 5 miles for consumer drones and 10 miles for higher-quality prosumer drones although they can see out further than this (Drone Pilot Ground School, 2024). The information collected during the flight is sent to a cloud-based AI platform that uses this information along with the motorcyclist's preferences to find the ideal route as the output. The images from the drone and the output from the AI system trickles down through the message broken and then out to both the motorcyclist's navigation system on his motorcycle if applicable and the motorcyclist's mobile app. Finally, the motorcycle can choose to accept or deny this route and further decide to send the analysis from the distributed system to fellow motorcyclists nearby that have the same app installed.

Conclusion

This DMS has been designed to provide motorcyclists with a safer, more accurate, and more personalized way to find the perfect route on a motorcycle ride. Although there is a restriction in drone range when it comes to extensive routes, the system is great for smaller to medium size trips or portions of a long route as it solves some of the issues presented by traditional navigation systems and is tailored to fit the wants and needs of motorcyclists.

References

- Dukowitz, Z. (2024, September 26). How Far Can a Drone Fly? Drone Flight Ranges & Legal Limits. Drone Pilot Ground School. <https://www.dronepilotgroundschool.com/how-far-can-a-drone-fly/>
- Swiss. (2024, October 29). Study finds GPS systems often mislead drivers. Techxplore.com; Tech Xplore. <https://techxplore.com/news/2024-10-gps-traffic-chaos-personal-responsibility.html>

Appendix

Header and Communicator Setup

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define MASTER 0
#define TAG 1

// Define communicators to group related entities
MPI_Comm appSyncComm, brokerComm, droneAComm;

// Summary: Sets up required packages and defines comm groups to be used for the following entities
```

Mobile App

```
void MobileApp() {
    char routeRequest[256] = "Requesting Route";
    char bestRoute[256];

    // Unicast send to Broker
    MPI_Send(routeRequest, 256, MPI_CHAR, 1, TAG, MPI_COMM_WORLD);

    // Receive bestRoute from within same communicator
    MPI_Recv(bestRoute, 256, MPI_CHAR, 1, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    // Broadcast bestRoute selection to group riders within appSyncComm
    MPI_Bcast(bestRoute, 256, MPI_CHAR, MASTER, appSyncComm);
}

// Summary: Mobile app sends routeRequest entered from the user to the MessageBroker, receives the
computed bestRoute from the MessageBroker once the data moves through the DMS, and broadcasts
this bestRoute to all other riders
```

Message Broker

```
void MessageBroker() {
    char routeRequest[256];
    char bestRoute[256];

    // Acts as a central controller, receiving all requests from motorcyclist
    MPI_Recv(routeRequest, 256, MPI_CHAR, 0, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    // Non-blocking Send to Drone for data capture
    MPI_Isend(routeRequest, 256, MPI_CHAR, 2, TAG, MPI_COMM_WORLD, MPI_REQUEST_NULL);

    // Mixed messages, reacting to Drone and AI data autonomously
    MPI_Recv(bestRoute, 256, MPI_CHAR, 3, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    // Packed/Unpacked Message – Broker sends optimized route to multiple nodes
    MPI_Send(bestRoute, 256, MPI_CHAR, 0, TAG, MPI_COMM_WORLD); // Send to Mobile App
    MPI_Send(bestRoute, 256, MPI_CHAR, 4, TAG, MPI_COMM_WORLD); // Send to Navigation System
}

// Summary: Message Broker acts as a central controller that receives routeRequest from the Mobile
App as well as the processed bestRoute from the AIProcessing. It utilizes a non-blocking send to the
drone to go out and capture data while it sends the bestRoute to the Mobile App and Navigation System
throughout the communicators
```

Drone System

```
void DroneSystem() {
    // Instance of variable data/message size
    char rawData[512];
    char routeRequest[256];

    // Receive for route request
    MPI_Recv(routeRequest, 256, MPI_CHAR, 1, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```



```
// Internal Processing (captures route data)
9print(rawData, "Captured Drone Data");

// Autonomous Data Transmission to AI
MPI_Send(rawData, 512, MPI_CHAR, 3, TAG, droneAComm);
}

// Summary: Drone System receives routeRequest from Message Broker and captures data to be
processed internally. This rawData is then sent through the droneAComm to be interpreted by the AI
Processing
```

AI Processing

```
void AIProcessing() {
    char rawData[512];
    char bestRoute[256] = "Computed Best Route";

    // Receive from Drone
    MPI_Recv(rawData, 512, MPI_CHAR, 2, TAG, droneAComm, MPI_STATUS_IGNORE);

    // Internal computation of best route
    9print(bestRoute, "AI Optimized Route");

    // Computed bestRoute sent to Broker through brokerComm
    MPI_Send(bestRoute, 256, MPI_CHAR, 1, TAG, brokerComm);
}

// Summary: AI Processing receives rawData from Drone System that then computes the bestRoute
(smaller size message than rawData). The bestRoute is finally sent to the centralized Message Broker
through the brokerComm
```

Navigation System

```
void NavigationSystem() {
    char bestRoute[256];

    // Receive from Broker
    MPI_Recv(bestRoute, 256, MPI_CHAR, 1, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    // Display route to screen
    printf("Displaying new route: %s", bestRoute);
}

// Summary: Navigation System is a reactive node that simply receives and displays bestRoute computed
from the rest of the system
```

Initializers

```
int main(int argc, char* argv[]) {
    // Setup of program and MPI_COMM_WORLD
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Establishing communicator groups
    MPI_Comm_split(MPI_COMM_WORLD, rank < 2, rank, &appSyncComm);
    MPI_Comm_split(MPI_COMM_WORLD, rank < 4, rank, &brokerComm);
    MPI_Comm_split(MPI_COMM_WORLD, rank >= 2, rank, &droneAIComm);

    // Assigning ranks
    if (rank == 0) MobileApp();
    else if (rank == 1) MessageBroker();
    else if (rank == 2) DroneSystem();
    else if (rank == 3) AIProcessing();
    else if (rank == 4) NavigationSystem();
}
```

```
else if (rank == 5) GroupRideSync();
```

```
// Close program
```

```
MPI_Finalize();
```

```
return 0;
```

```
}
```

```
// Summary: Sets up initializer and finalize functions as well as rank and communicator groupings
```